

SW Testing Techniques

Farjana Parvin
Lecturer
Dept. of CSE, RUET

Testing Objectives

- Testing is a process of executing a program with the intent of finding an error.
- A good test case is one that has a high probability of finding an as-yet-undiscovered error.
- A successful test is one that uncovers an as-yet-undiscovered error.

Test Case Design

Any engineered product can be tested in one of two ways:

- Knowing the specified function that a product has been designed to perform, tests can be conducted that demonstrate each function is fully operational while at the same time searching for errors in each function;
- Knowing the internal workings of a product, tests can be conducted to ensure that "all gears mesh," that is, internal operations are performed according to specifications and all internal components have been adequately exercised.
- The first test approach is called black-box testing and the second, white-box testing.

Black-Box Testing

- Also Known As (AKA) behavioral testing, specification testing, i/p-o/p testing.
- Focuses on functional requirements of the SW
- It is a test case design method.
- Testers evaluate the functionality of the SW under test without looking at the internal code structure.
- Can be applied to Unit, integration, System and Acceptance testing.
- Black-box testing attempts to find errors in the following categories:
 - Incorrect or missing functions
 - interface errors
 - errors in data structures or external data base access
 - behavior or performance errors
 - initialization and termination errors

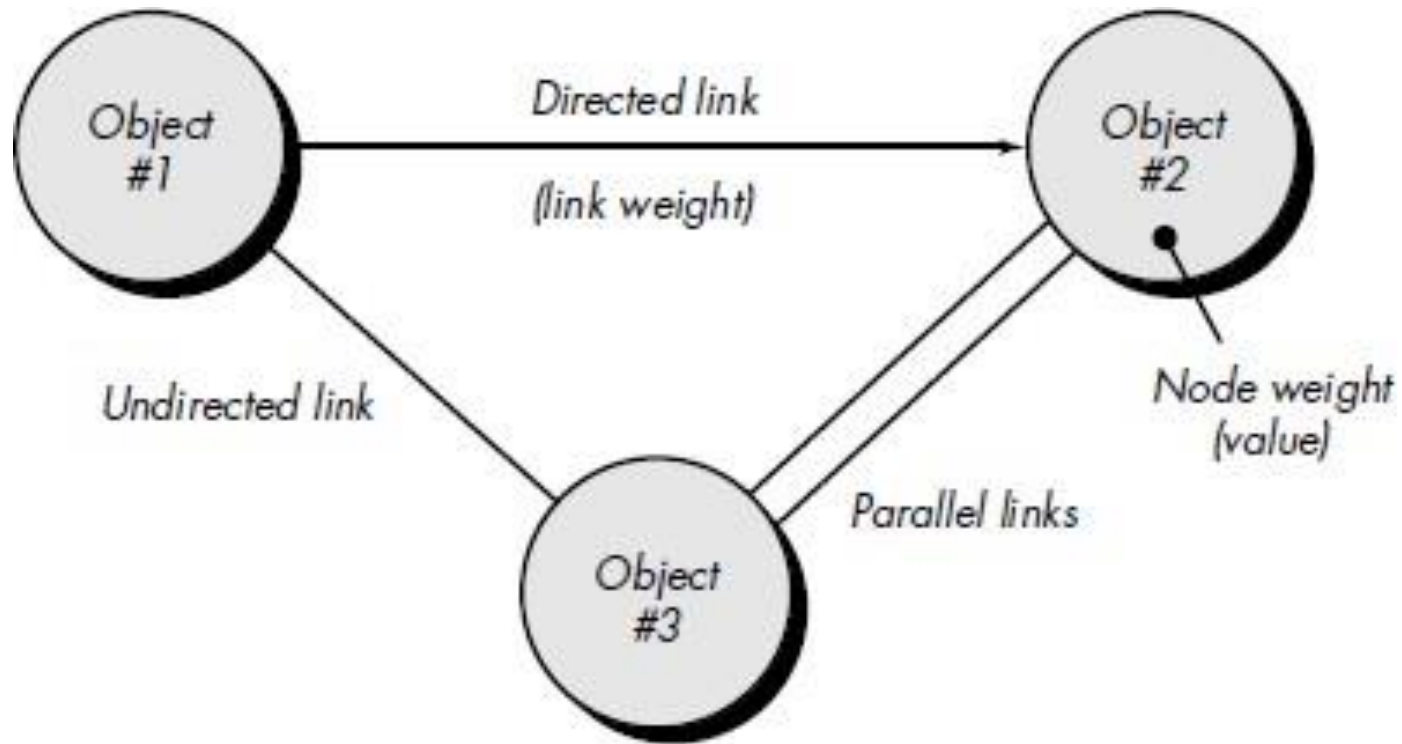
Graph-Based Testing Methods

- The first step in black-box testing is to understand the objects that are modeled in software and the relationships that connect these objects.
- Created a *graph*—
 - ✓ a collection of *nodes* that represent objects;
 - ✓ *links* that represent the relationships between objects;
 - ✓ *node weights* that describe the properties of a node (e.g., a specific data value or state behavior);
 - ✓ *link weights* that describe some characteristic of a link

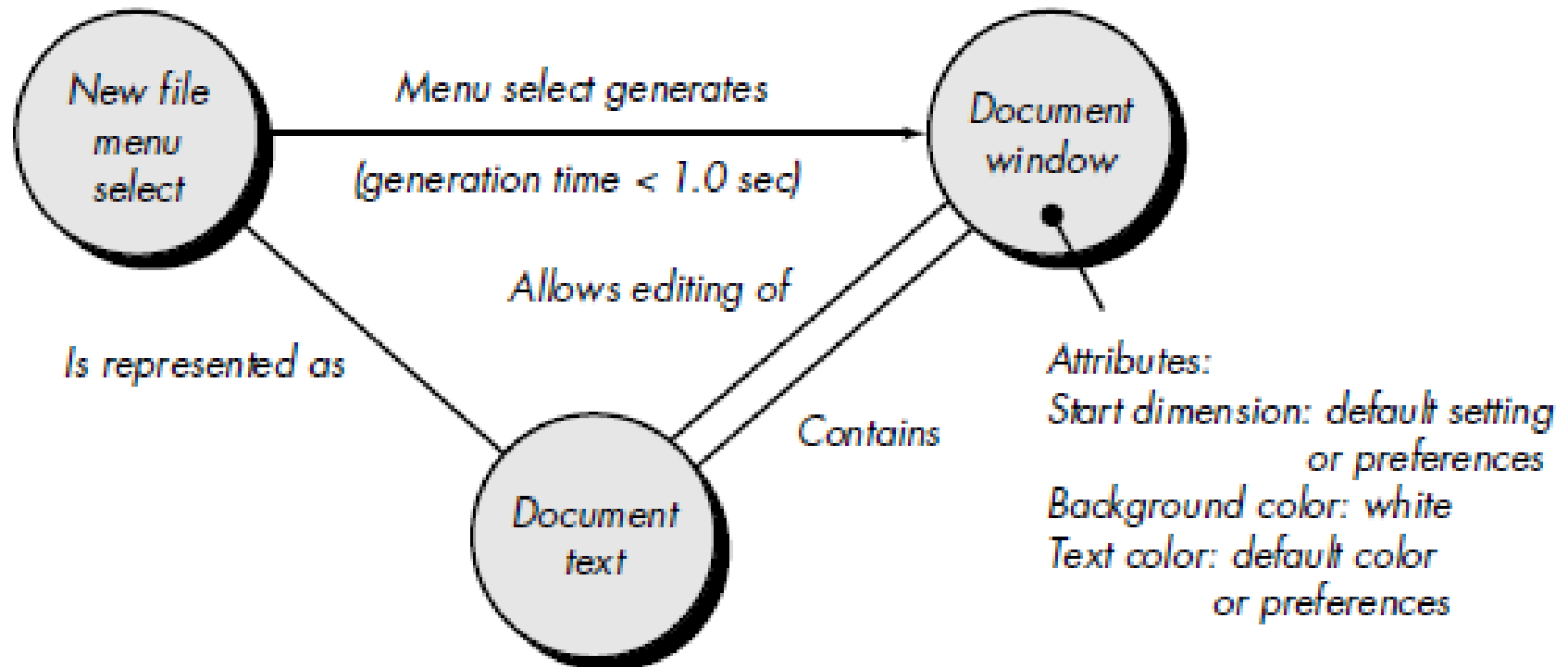
Graph-Based Testing Methods...

- ✓ A *directed link* (represented by an arrow) indicates that a relationship moves in only one direction.
- ✓ A *bidirectional link*, also called a *symmetric link*, implies that the relationship applies in both directions.
- ✓ *Parallel links* are used when a number of different relationships are established between graph nodes.

Graph-Based Testing Methods...



Graph-Based Testing Methods...



White-Box Testing

- AKA glass-box testing, clear-box testing, structural testing.
- It is based on application's internal code structures.
- It is a test-case design method to test SW.
- Internal perspective of the system as well as programming skills are used to design test cases.
- It is used in unit testing.
- It focuses on-
 - All independent paths within a module which have been exercised at least once.
 - Exercise all logical decisions on their true or false sides.
 - Executes all loops at their boundaries and within their operational bounds.
 - Exercise internal data structures to ensure their validity

Basis Path Testing

- *Basis path testing* is a white-box testing technique
- Enables the test case designer to derive a logical complexity measure (Cyclomatic Complexity) of a procedural design
- Use that measure as a guide for defining a basis set of independent execution paths
- Test cases are derived to exercise the basis set

Flow Graph Notation

- The flow graph depicts logical control flow
- Each circle, called a *flow graph node*, represents one or more procedural statements.
- A sequence of process boxes and a decision diamond can map into a single node.
- The arrows on the flow graph, called *edges* or *links*, represent flow of control and are analogous to flowchart arrows.
- An edge must terminate at a node, even if the node does not represent any procedural statements.
- Areas bounded by edges and nodes are called *regions*.
- When counting regions, we include the area outside the graph as a region

Flow Graph Notation...

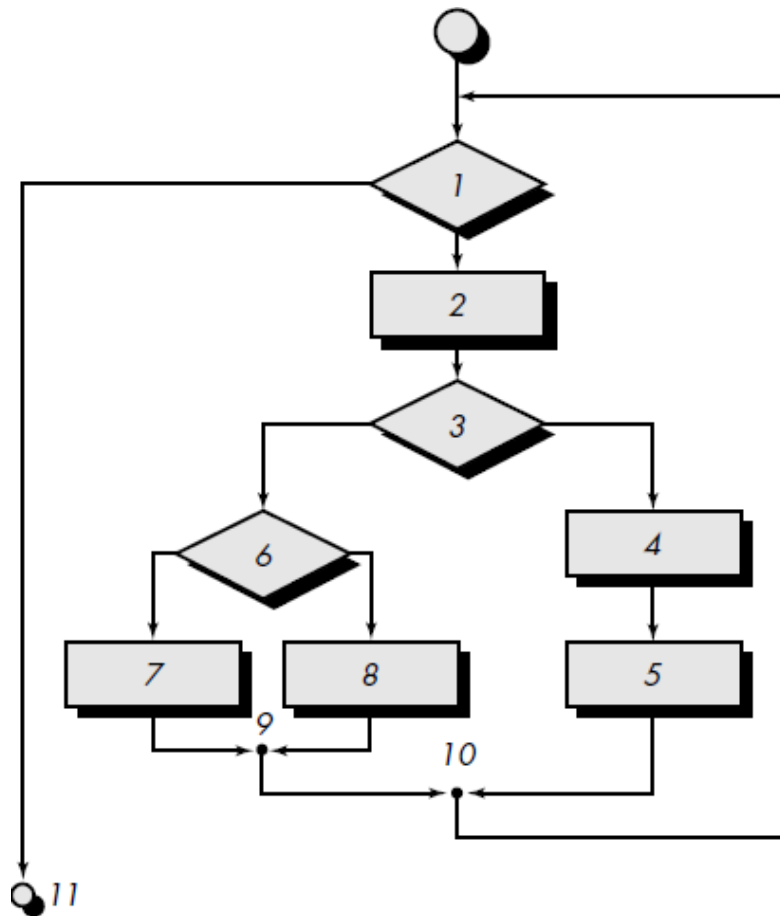


Fig - 01

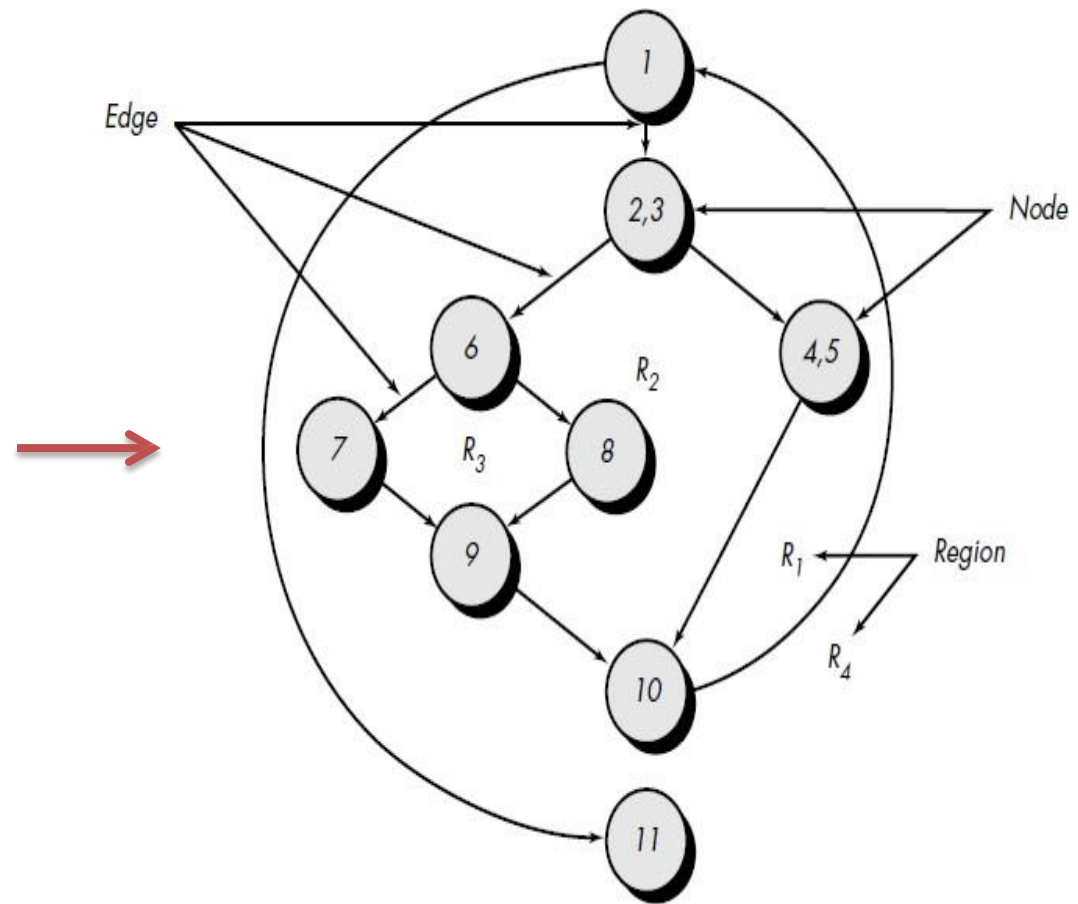


Fig - 02

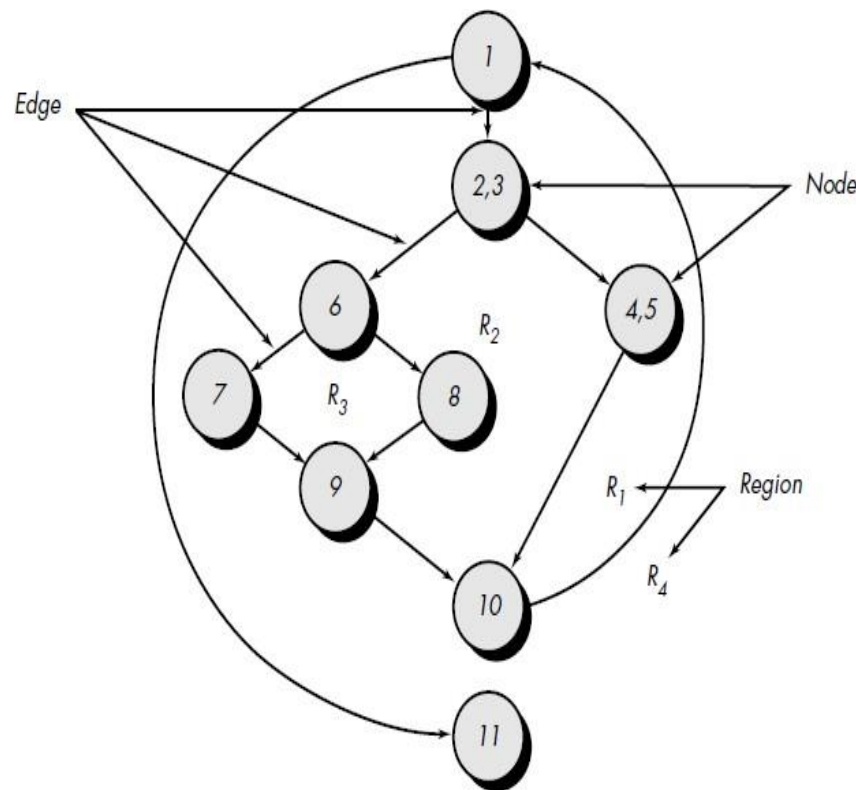
Cyclomatic Complexity

- *Cyclomatic complexity* is a software metric that provides a quantitative measure of the logical complexity of a program.
- When used in the context of the basis path testing method, the value computed for cyclomatic complexity defines the number of independent paths in the basis set of a program.
- Provides us with an upper bound for the number of tests that must be conducted to ensure that all statements have been executed at least once.
- An *independent path* is any path through the program that introduces at least one new set of processing statements or a new condition.

Cyclomatic Complexity...

- Complexity is computed in one of three ways:
 1. The number of regions of the flow graph correspond to the cyclomatic complexity.
 2. Cyclomatic complexity, $V(G)$, for a flow graph, G , is defined as $V(G) = E - N + 2$, where E is the number of flow graph edges, N is the number of flow graph nodes.
 3. Cyclomatic complexity, $V(G)$, for a flow graph, G , is also defined as $V(G) = P + 1$ where P is the number of predicate nodes contained in the flow graph G .

Cyclomatic Complexity Example



1. The flow graph has four regions.
2. $V(G) = 11 \text{ edges} - 9 \text{ nodes} + 2 = 4.$
3. $V(G) = 3 \text{ predicate nodes} + 1 = 4.$

Deriving Test Cases

1. Using the design or code as a foundation, draw a corresponding flow graph.
2. Determine the cyclomatic complexity of the resultant flow graph.
3. Determine a basis set of linearly independent paths.
4. Prepare test cases that will force execution of each path in the basis set.

Example

PROCEDURE average;

- * This procedure computes the average of 100 or fewer numbers that lie between bounding values; it also computes the sum and the total number valid.

INTERFACE RETURNS average, total.input, total.valid;

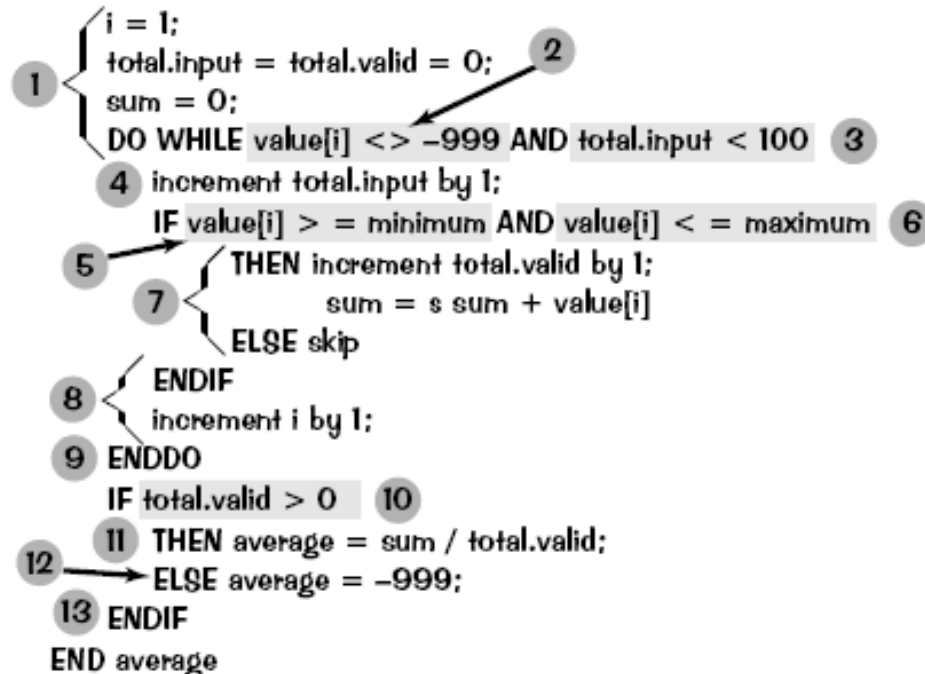
INTERFACE ACCEPTS value, minimum, maximum;

TYPE value[1:100] IS SCALAR ARRAY;

TYPE average, total.input, total.valid;

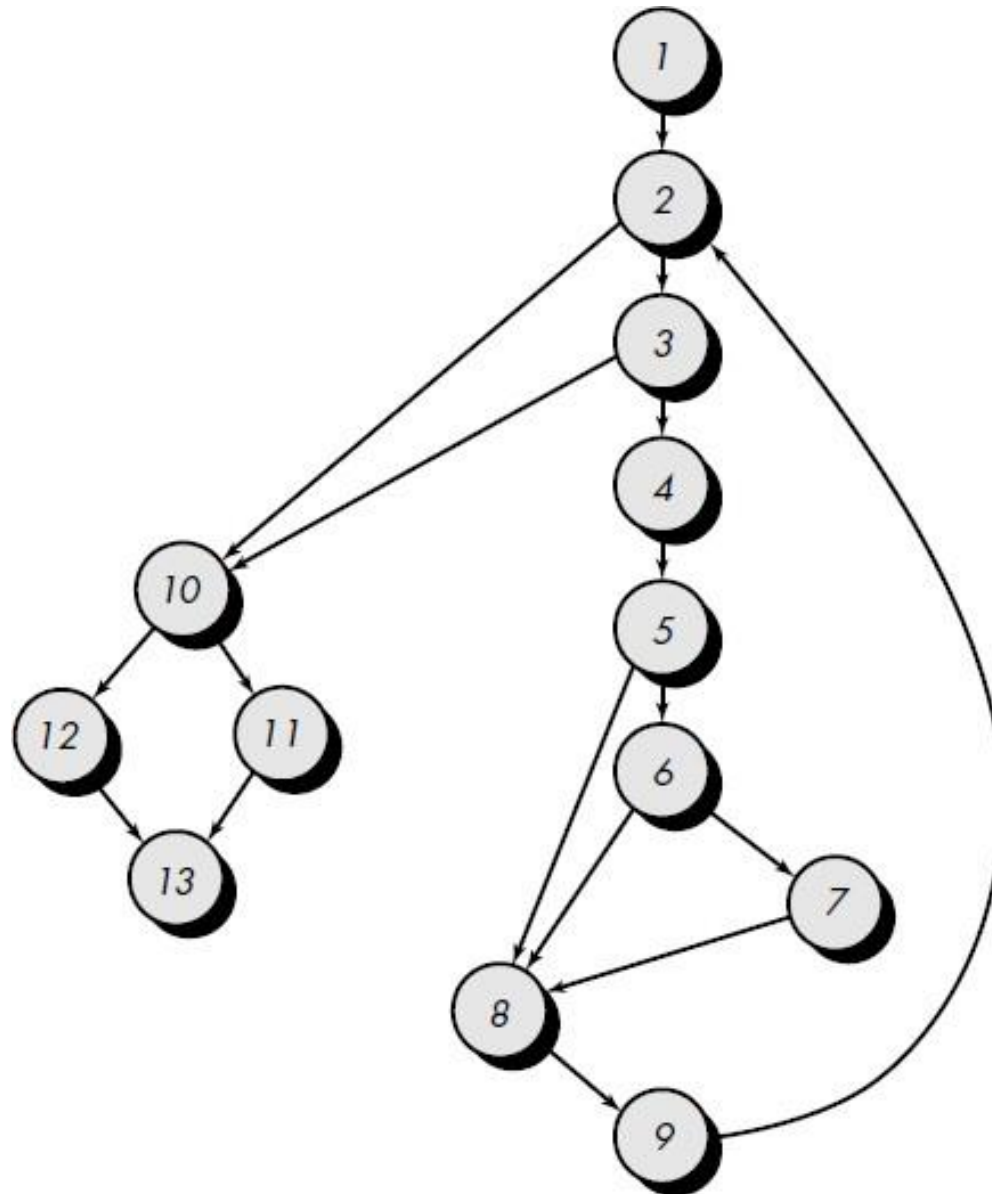
minimum, maximum, sum IS SCALAR;

TYPE i IS INTEGER;



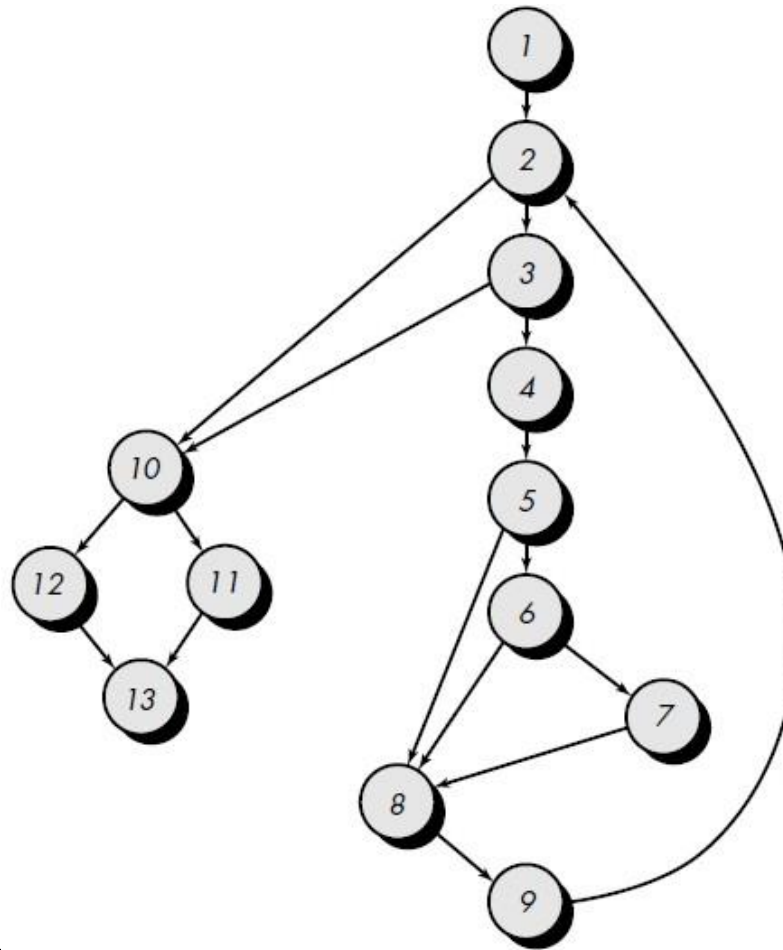
Example...

Step - 1



Example...

Step - 2



$V(G) = 6$ regions

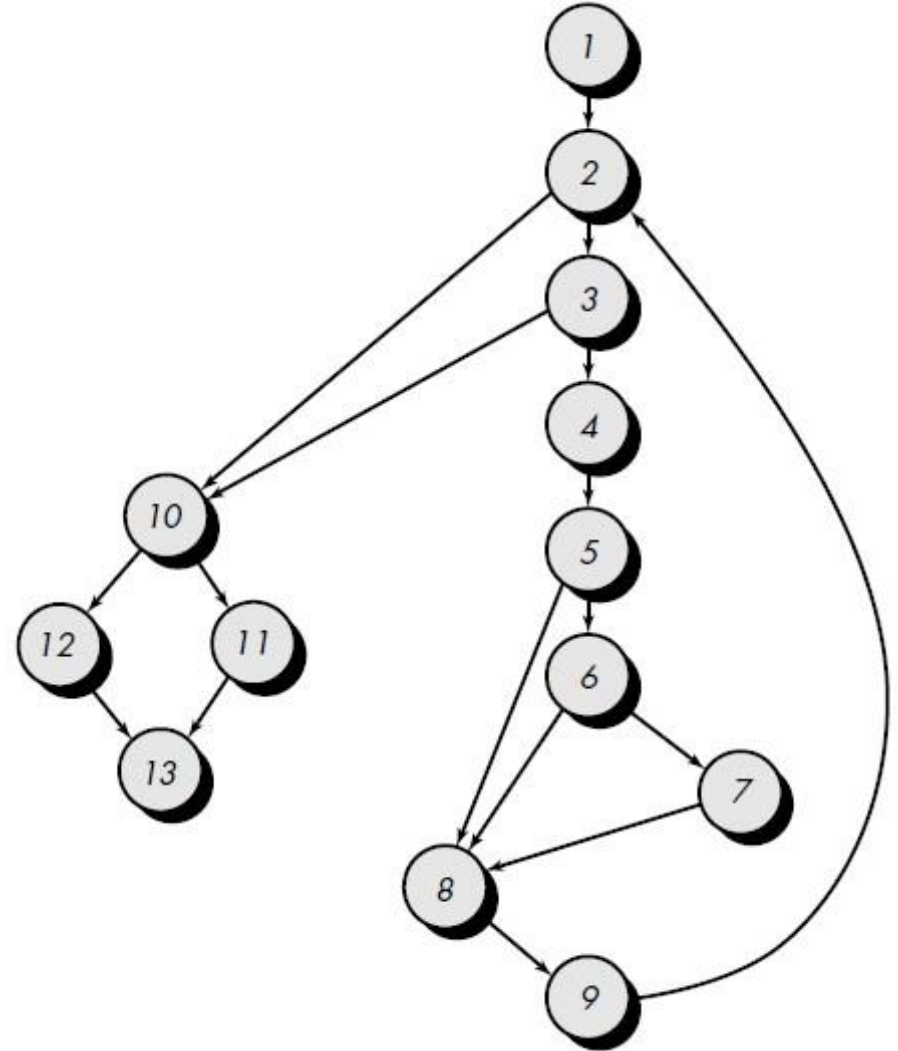
$V(G) = 17 \text{ edges} - 13 \text{ nodes} + 2 = 6$

$V(G) = 5 \text{ predicate nodes} + 1 = 6$

Example...

Step - 3

- path 1: 1-2-10-11-13
- path 2: 1-2-10-12-13
- path 3: 1-2-3-10-11-13
- path 4: 1-2-3-4-5-8-9-2-...
- path 5: 1-2-3-4-5-6-8-9-2-...
- path 6: 1-2-3-4-5-6-7-8-9-2-...



Example...

Step - 4

Path 1 test case:

value(k) = valid input, where $k < i$ for $2 \leq i \leq 100$

value(i) = -999 where $2 \leq i \leq 100$

Expected results: Correct average based on k values and proper totals.

Note: Path 1 cannot be tested stand-alone but must be tested as part of path 4, 5, and 6 tests.

Path 2 test case:

value(1) = -999

Expected results: Average = -999 ; other totals at initial values.

Path 3 test case:

Attempt to process 101 or more values.

First 100 values should be valid.

Expected results: Same as test case 1.

Path 4 test case:

value(i) = valid input where $i < 100$

value(k) < minimum where $k < i$

Expected results: Correct average based on k values and proper totals.

Example...

Step - 4...

Path 5 test case:

value(i) = valid input where $i < 100$

value(k) > maximum where $k \leq i$

Expected results: Correct average based on n values and proper totals.

Path 6 test case:

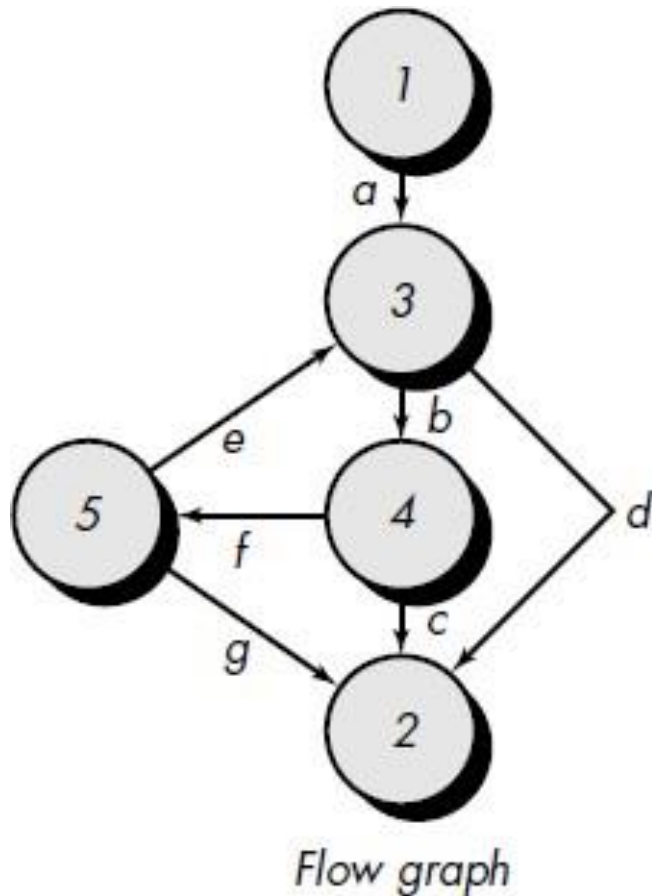
value(i) = valid input where $i < 100$

Expected results: Correct average based on n values and proper totals.

Graph Matrices

- To develop a software tool that assists in basis path testing, a data structure, called a *graph matrix*, can be quite useful.
- A *graph matrix* is a square matrix whose size (i.e., number of rows and columns) is equal to the number of nodes on the flow graph.

Graph Matrices...



Connected to node		1	2	3	4	5
Node		1	2	3	4	5
1				a		
2						
3			d		b	
4			c			f
5			g	e		

Graph matrix

Graph Matrices.....

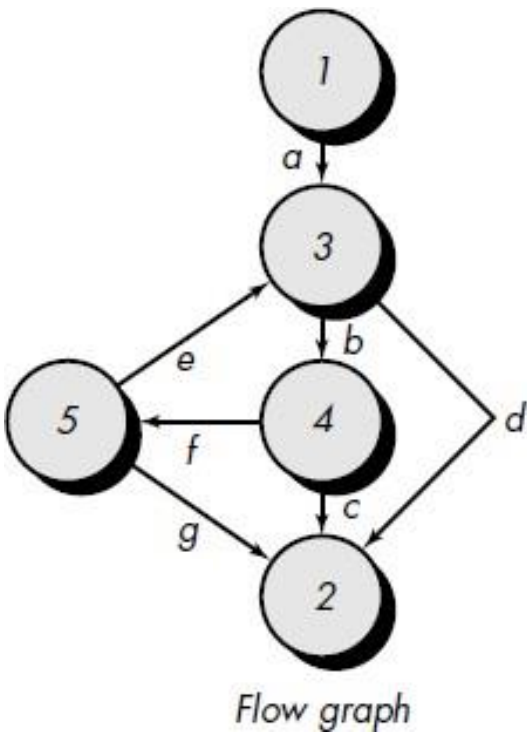
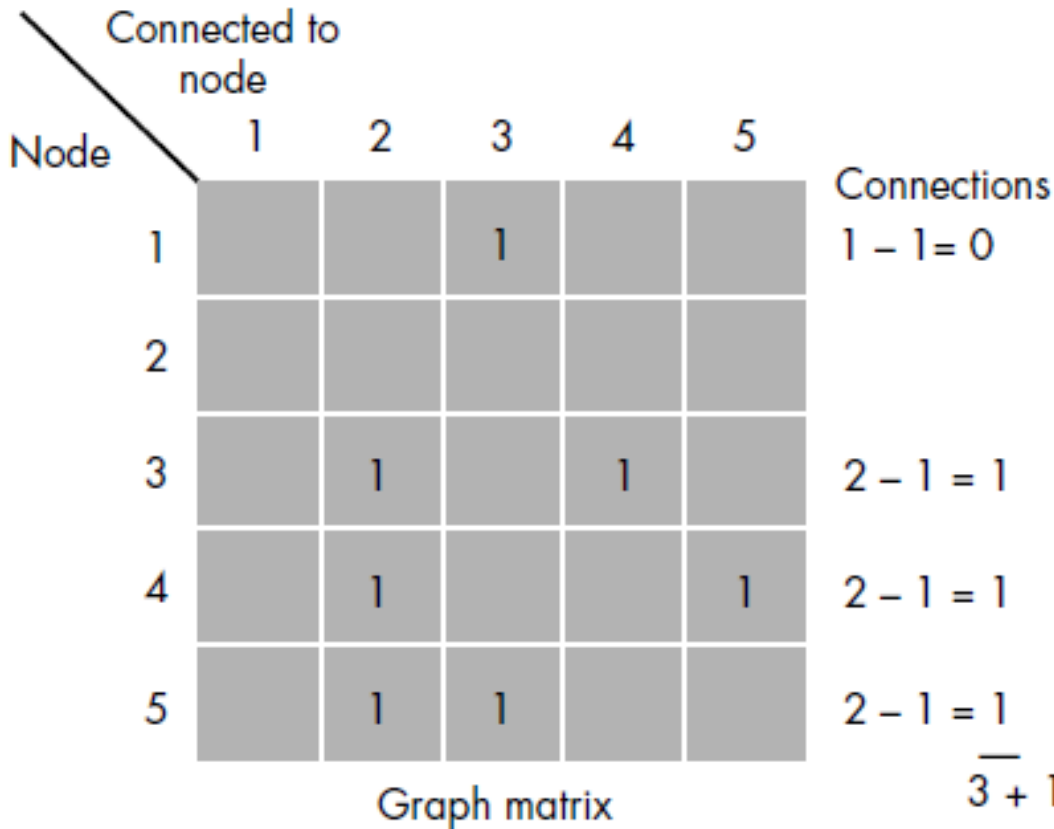
- To this point, the graph matrix is nothing more than a tabular representation of a flow graph. However, by adding a link weight to each matrix entry, the graph matrix can become a powerful tool for evaluating program control structure during testing. The link weight provides additional information about control flow. In its simplest form, the link weight is 1 (a connection exists) or 0 (a connection does not exist).

Graph Matrices.....

But link weights can be assigned other, more interesting properties:

- The probability that a link (edge) will be executed.
- The processing time expended during traversal of a link.
- The memory required during traversal of a link.
- The resources required during traversal of a link.

Graph Matrices...



Cyclomatic complexity

Thanks